



TED UNIVERSITY
Faculty of Engineering

CMPE 491/SENG491

Senior Project I

LLM Valley

High Level Design Report

TEAM 21

30.12.25

by

Yusuf Sadi Pesen

Deniz Şeker

Egemen Asım Ersoy

Reyhan Sena Çimen

Supervisor:

Dr.Emin Kuğu

Jury Members:

Dr. Ulaş Güleç

Dr. Eren Ulu

1. Introduction.....	3
1.1 Purpose of the System	3
1.2 Design Goals	3
1.3 Definitions, Acronyms, and Abbreviations.....	3
1.4 Overview	3
2. Current Software Architecture.....	4
3. Proposed Software Architecture.....	5
3.1 Overview	5
3.2 Subsystem Decomposition	7
3.2.1 Core Game Subsystem	7
3.2.2 AI & Dialogue Subsystem	7
3.2.3 Player & Interaction Subsystem	8
3.2.4 Economy & Environment Subsystem.....	8
3.2.5 UI Subsystem.....	8
3.3 Hardware/Software Mapping	8
3.4 Persistent Data Management	8
3.5 Access Control and Security	9
3.6 Global Software Control	9
3.7 Boundary Conditions.....	9
4. Subsystem Services	10
4.1 AI & Dialogue Services.....	10
4.2 Farming Services.....	11
4.3 Trading Services	11
5. Glossary	13
6. References (Everybody)	14

1. Introduction

1.1 Purpose of the System

The purpose of the LLM Valley system is to provide a narrative-driven life simulation demo that integrates Large Language Model (LLM) technology to drive dynamic Non-Player Character (NPC) conversations. Unlike traditional games with pre-determined dialogue context trees, this system allows players to interact with NPCs using natural language with text input, creating a unique and interesting experience for players. The system serves as a proof of concept to assess the feasibility, stability, and player utility of LLM-powered interactions within a real-time gameplay loop that includes farming and trading.

1.2 Design Goals

Based on the analysis and specifications, the system design prioritizes the following goals:

- **Performance (Low Latency):** The system is designed to fetching and displaying LLM responses within 5 seconds in 90% of cases to maintain gameplay flow.
- **Modularity:** The architecture decouples the core game logic (farming, movement) from the AI service layer. This ensures that the LLM API (DeepSeek/Gemini) can be switched or updated without refactoring the entire codebase.
- **Reliability:** The system must handle network failures or API timeouts gently by using local fallback mechanisms (e.g., "Thinking..." or default responses) to prevent crashes.
- **Resource Efficiency:** The game is optimized to run on mid-range consumer hardware (Windows 10+, 2018+ PCs) without requiring heavy local machine learning inference, as processing is offloaded to the cloud.
- **Ethical Safety:** The design includes input/output filtering to ensure content remains appropriate and aligns with ethical guidelines.

1.3 Definitions, Acronyms, and Abbreviations

- **LLM (Large Language Model):** AI model used to generate human-like text responses.
- **NPC (Non-Player Character):** Characters controlled by the system (AI) rather than the player.
- **Prefab:** A reusable, pre-configured object in Unity (e.g., NPC, Crop Plot).
- **API (Application Programming Interface):** The interface used to communicate with the remote LLM service (DeepSeek/Gemini).
- **Gameplay Loop:** The repetitive cycle of farming, trading, talking, and sleeping.

1.4 Overview

The LLM Valley system includes three significant components. We call them the Game Client, the AI Integration Module, and the Simulation Logic.

The Game Client is built on the Unity Engine and serves as the user interface where the player navigates the 3D world, manages inventory, and inputs free-text dialogue.

The AI Integration Module utilizes remote Large Language Model APIs (DeepSeek/Gemini) for the generation of dynamic NPC conversations. This module is responsible for sending user inputs to the cloud and parsing the returned text to drive character behavior and relationship stats, replacing traditional static dialogue trees.

The Simulation Logic manages the real-time gameplay loop. This includes tracking crop growth cycles, calculating economic values for trading, and handling persistent storage of game data (save/load) using local JSON serialization.

2. Current Software Architecture

As LLM Valley is a standalone demo developed from scratch for this senior project, there is no legacy software architecture to maintain or refactor. The project is a greenfield development. However, it leverages the Unity Engine's component-based architecture as its foundational framework. In addition to this, preliminary experimental trials are being conducted locally on a PC, utilizing the DeepSeek model with Ollama. This allows us to test potential outcomes and responses within relevant interactions.

3. Proposed Software Architecture

3.1 Overview

The proposed software architecture follows a Client-Server model integrated with a Component-Based game architecture. The architecture we made for our game's system is carefully designed to meet all the functional and non-functional requirements stated in our Specifications and analysis reports. This architecture has a high connection with the game logic and coupling between the game client and the External AI services. It also gives high priority to the real-time performance of the gameplay loop while still maintaining robust communication for the AI features.

Considering the unique hybrid nature of our application which combines simulation with cloud-based user interface. It is described that the architecture follows a client-server model integrated with a component-based game structure native to unity game engine. This allows for a clear separation of rendering multiple and many resources and local game logic of the computationally heavy LLM features.

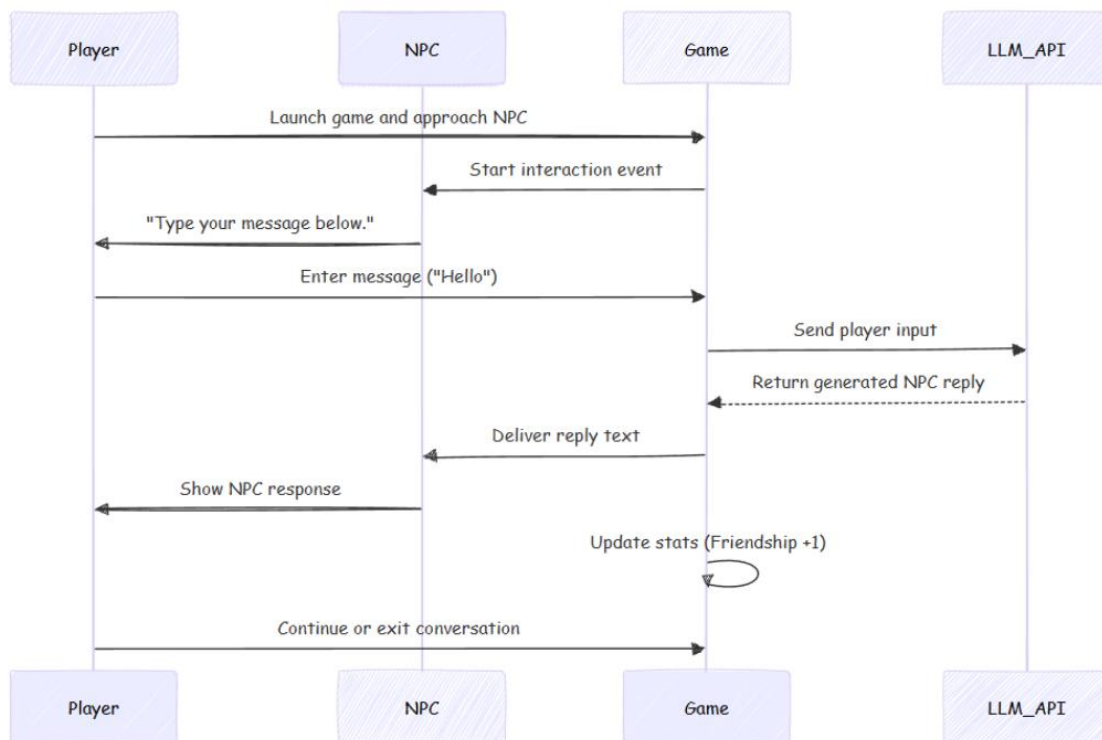


Figure 1 : High-Level Client-Server Architecture

The advantages that this hybrid pattern provides for the LLM Valey system include:

- **Scalability:** The AI logic is decoupled from the client, allowing the backend model (Gemini / DeepSeek) to be upgraded or switched without requiring patches to the client.
- **Performance:** Offloading to heavy matrix multiplications of LLM inference. Game must remain playable on standard consumer hardware.
- **Modularity:** Each subsystem can be developed and tested in Prefab isolation before getting integrated.

Key Technologies:

- Game Engine: Unity 2022 LTS
- Language: C#
- Data Transport: HTTPS using JSON payloads
- AI Backend: REST APIs provided by DeepSeek or Google Gemini

3.2 Subsystem Decomposition

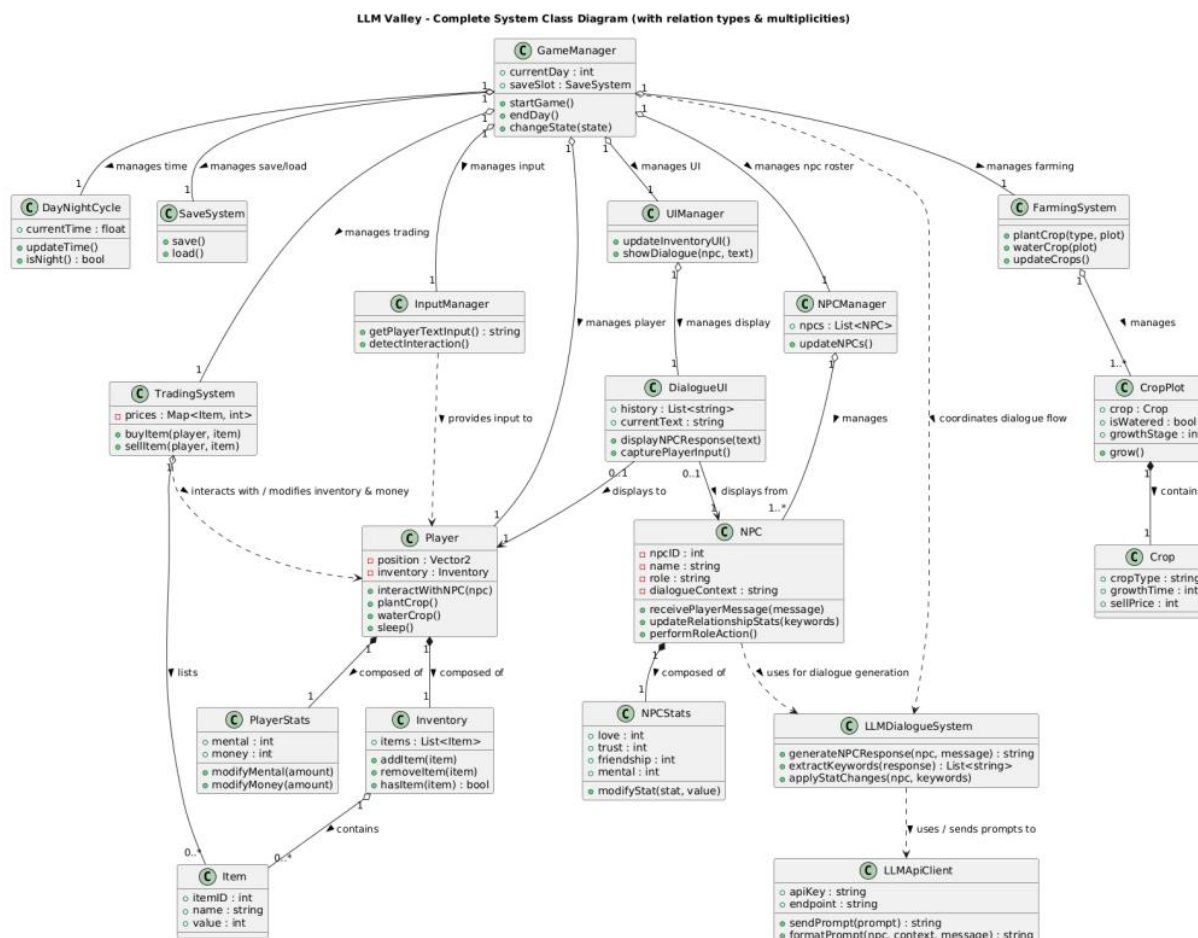


Figure 2: Subsystem Decomposition and Component Relationships

Based on the Class Diagram provided in the Analysis Report (Figure 4), the system is decomposed into the following logical subsystems:

3.2.1 Core Game Subsystem

Responsible for the main game loop, time management, and global state.

- **Components:** GameManager, DayNightCycle, SaveSystem.
- **Responsibility:** Manages the transition between days, triggers save/load operations and coordinates other subsystems.

3.2.2 AI & Dialogue Subsystem

Responsible for all interactions involving text generation and NPC behavior.

- **Components:** LLMDialogueSystem, LLMApiClient, NPCManager, NPC.

- **Responsibility:** Formats user input into prompts, sends requests to the API, parses the JSON response, and applies stat updates (Love, Trust) based on keyword analysis.

3.2.3 Player & Interaction Subsystem

Handles user input and character representation.

- **Components:** Player, InputManager, Inventory, PlayerStats.
- **Responsibility:** Captures keyboard/mouse input, manages the player's position, and holds inventory data.

3.2.4 Economy & Environment Subsystem

Manages the simulation aspects of the game.

- **Components:** FarmingSystem, CropPlot, TradingSystem.
- **Responsibility:** Tracks crop growth stages, handles watering logic, and calculates item prices for buying/selling.

3.2.5 UI Subsystem

Manages visual feedback to the player.

- **Components:** UIManager, DialogueUI.
- **Responsibility:** Displays the dialogue history, stat bars, and trading interfaces.

3.3 Hardware/Software Mapping

The system uses a straightforward mapping strategy optimized for standard PC hardware.

Node	Software Component	Hardware/Platform
Client Node	Unity Executable (Game Logic, UI, Physics)	PC: Windows 10/11, Keyboard & Mouse, Internet Connection.
Data Layer	Local JSON File (Save Data)	Local Storage: HDD/SSD on the Client PC.
Server Node	DeepSeek/Gemini API (LLM Inference)	Cloud Server: Remote infrastructure accessed via HTTPS.

3.4 Persistent Data Management

- **Strategy:** The system utilizes File-Based Persistence.
- **Implementation:** A `SaveSystem` class serializes key game data (Current Day, Inventory List, NPC Relationship Stats, Crop States) into a local JSON file.

- **Rationale:** Since this is a single-player demo without multiplayer features, a complex database (SQL/NoSQL) is unnecessary. JSON provides a lightweight, human-readable format that is natively supported by Unity's `JsonUtility`.

3.5 Access Control and Security

- **API Security:** API keys for the LLM service are stored securely within the application build (or environment variables during development) to prevent exposure. Communication is encrypted via HTTPS.
- **User Privacy:** The system adheres to a "No-Storage" policy for user data. Player inputs are sent to the LLM solely for context generation and are not stored permanently on any server managed by Team 21.
- **Content Safety:** The prompt engineering layer includes instructions to filter harmful or explicit content, ensuring the output adheres to the ACM Code of Ethics.

3.6 Global Software Control

The global control flow is Event-Driven, managed by the Unity Engine's main loop.

1. **Input Handling:** The `InputManager` listens for user actions (keystrokes) every frame.
2. **State Machine:** The dialogue system operates on a finite state machine (Figure 5 in Analysis Report): Idle -> Input Listening -> Processing (API Call) -> Response Generation -> Termination.
3. **Asynchronous Operations:** Calls to the `LLMApiClient` are asynchronous (coroutines or `async/await`) to prevent freezing the UI while waiting for the 3-5 second API response.

3.7 Boundary Conditions

- **Startup:** The `GameManager` initializes subsystems and loads data from the JSON file if it exists.
- **API Failure/Timeout:** If the LLM API fails to respond within the timeout threshold, the system triggers an `Error Handling` state, displaying a fallback message (e.g., "Sorry, I didn't catch that") and logging the error locally, ensuring the game does not crash.
- **Shutdown:** On exit, the `OnApplicationQuit` event triggers the `SaveSystem` to write the current state to the disk.

4. Subsystem Services

This section provides a detailed breakdown of the services offered by each subsystem, defining the logic, data flow, and specific operations required to sustain the LLM Valley gameplay loop.

4.1 AI & Dialogue Services

The AI & Dialogue Subsystem is the core innovation of the project, replacing static dialogue trees with a dynamic, LLM-powered communication layer.

- **SendPrompt(npc, context, message):**

Logic: This service constructs a comprehensive "system prompt" by concatenating the NPC's persona (role, name, and personality traits), current environmental context (time of day, current day count), and the player's free-text input.

Asynchronous Execution: To prevent the game UI from lagging for the API process duration (expectedly 3–5 seconds), this command is executed either as a coroutine or async task.

Fallback Integration: If the LLMApiClient fails to return a response within the designated time limit, this service calls for the fallback mechanism to provide a "Thinking..." or "I didn't catch that" message.

- **ExtractKeywords(response):**

Pattern Matching: After receiving the original string from the LLM, this service examines the text to find specific tags or keywords (e.g., "[Trust+1]", "happy", "angry").

Data Translation: It translates qualitative AI-generated sentiment into quantitative values that the C# game logic can process.

- **UpdateRelationship(npc, stats):**

State Persistence: Modifies the NPCStats object in memory, updating variables for Love, Trust, Friendship, and Mental health.

Tone Adjustment: These updated values are fed back into the next SendPrompt call, ensuring the NPC's tone evolves based on the history of interactions.

4.2 Farming Services

The Farming System manages the lifecycle of crops and the player's interaction with the physical environment.

- **PlantCrop(type, plot):**

Validation: Checks the player's Inventory for the needed seed type and makes sure that the CropPlot is currently empty.

Initialization: Takes the plot's coordinates, instantiates the crop prefab there, and sets the initial growthStage to zero.

- **WaterCrop(plot):**

State Toggle: Updates the isWatered boolean of the targeted CropPlot to true.

Dependency: Watering is a prerequisite for the growth cycle to advance during the next sleep transition.

- **AdvanceGrowth():**

Global Update: Triggered by the DayNightCycle when the player sleeps.

Logic: Iterates through all plots; if isWatered is true, it increments growthStage. If a crop reaches its growthTime, it becomes harvestable.

Reset: Sets all plots back to isWatered = false for the new day.

4.3 Trading Services

The Trading Subsystem manages the game's economy, utilizing the LLM to create a more dynamic marketplace.

- **GetPrice(item, playerStats):**

Dynamic Valuation: Rather than using static prices, this service calculates the cost of seeds or the value of harvested crops based on the player's current relationship stats with the shopkeeper.

Prompting Logic: In specific scenarios, the system may even send the item data to the LLM to generate a custom price based on the "friendship level".

- **ExecuteTransaction(player, item, transactionType):**

Balance Verification: For "Buy" actions, it checks if PlayerStats.money is enough to go on with the transaction.

Inventory Management: For "Sell" actions, it decreases the item count in the Inventory and adds the calculated sell value to the player's account.

UI Feedback: Calls for the UIManager to tell the player if the transaction was a success or a failure.

5. Glossary

- **Tokenizer:** A preprocessing tool that splits text into pieces (tokens) used by the LLM.
- **Fallback Response:** A pre-written default message used when the AI service is unavailable. unavailable.
- **JsonUtility:** Unity's built-in library for handling JSON serialization.
- **Relationship Stats:** Numerical values representing Love, Trust, Friendship, and Mental states⁴⁶.
- **NPC:** Non Player Character Gameplay Loop: A repeating cycle of actions players perform during normal play (e.g., farm → trade → talk → sleep).
- **In-Game Day / Day Cycle:** A simulated time cycle where events reset or progress after a "sleep" action.
- **Stat System / Relationship Stats:** Variables representing emotional or social parameters (Love, Trust, Friendship, Mental).
- **Inventory System:** A structure that holds items the player owns.
- **Tokenizer:** A preprocessing tool that splits text into pieces (tokens) used by the LLM.
- **Fallback Response:** A default message given when the LLM API fails or times out.
- **Script / MonoBehaviour:** A C# class attached to Unity GameObjects to control behavior.
- **Prefab:** A reusable pre-configured object (NPC, crop plot, item, etc.) that can be instantiated in the game.
- **GameObject:** The basic building block of Unity scenes that can hold components such as sprites, colliders, and scripts.
- **Scene:** A collection of objects forming a game level.
- **UI Canvas:** Unity's system for rendering user interface elements.

6. References

- [1] TED University, Faculty of Engineering, "LLM Valley Project Specifications Report," Team 21, 2025.
- [2] TED University, Faculty of Engineering, "Analysis Report- Group 21-LLM Valley," Team 21, 2025.
- [3] IEEE Computer Society, "Software Engineering Code of Ethics," 47.
- [4] Unity Technologies, "Unity Manual and Scripting API," 202448.
- [5] Bernd Bruegge and Allen H. Dutoit, "Object-Oriented Software Engineering, Using UML, Patterns, and Java," 2nd Edition, Prentice-Hall, 2004.
- [6] L. Song, "LLM-Driven NPCs: Cross-Platform Dialogue System for Games and Social Platforms," 2025.
- [7] L. M. Csepregi, "The Effect of Context-aware LLM-based NPC Conversations on Player Engagement in Role-playing Video Games," Department of Architecture, Design and Media Technology, Aalborg University, Copenhagen, Denmark, 2024.
- [8] G. Zeng, "A review of AI-based game NPCs research," Applied and Computational Engineering, vol. 15, pp. 155–159, 2023.
- [9] Unity Technologies, "Unity Manual and Scripting API," Unity Documentation, 2024. [Online]. Available: <https://docs.unity3d.com/Manual/index.html>
- [10] Unity Technologies, "Using JSONUtility for Saving and Loading Data," Unity Documentation, 2023. [Online]. Available: <https://docs.unity3d.com/ScriptReference/JsonUtility.html>
- [11] ConcernedApe. Stardew Valley [Video game]. ConcernedApe, 2016. Available at: https://store.steampowered.com/app/413150/Stardew_Valley/
- [12] Sam Edwards. Travellers Rest [Video game]. Isolated Games, 2020. Available at: https://store.steampowered.com/app/1139980/Travellers_Rest/
- [13] Fair Games Studio. Necesse [Video game]. Fair Games Studio, 2019. Available at: <https://store.steampowered.com/app/1169040/Necesse/>
- [14] GameDeveloper.com, "Design Patterns of Farming and Life Simulation Games," 2020. [Online]. Available: <https://www.gamedeveloper.com/design>

[15] Object-Oriented Software Engineering, Using UML, Patterns, and Java, 2nd Edition, by Bernd Bruegge and Allen H. Dutoit, Prentice-Hall, 2004, ISBN: 0-13-047110-0.